

Campaign Recommendation Implementation And Deployment Guide

1. Project Objective

This project builds an ML-based campaign recommendation engine for EMI collections communication.

The system predicts the most suitable communication strategy for each loan account across the EMI window:

- 'D-5' to 'D+5'
- 'D' is excluded

The business rule is applied ****per day****:

- 'low' risk: up to '1' communication per day
- 'medium' risk: up to '2' communications per day
- 'high' risk: up to '3' communications per day

If the previous month has no campaign signal for a given day for a customer, that day is returned as '- '.

2. High-Level Architecture

The pipeline is organized into these stages:

1. Read raw communication history data
2. Filter only valid EMI records
3. Build channel-aware target labels
4. Build previous-month customer history features
5. Build previous-month day-availability features
6. Create time-based train, validation, and test splits
7. Train the recommendation model
8. Calibrate probabilities
9. Score candidate day/channel/hour combinations
10. Apply daily business quotas by risk
11. Export strategy outputs and summary reports

3. Repository Structure

- [README.md](/d:/aiml/campaign-recommenadtion/README.md)
Short project overview and quick-start commands.
- [config/default_config.json](/d:/aiml/campaign-recommenadtion/config/default_config.json)
Main configuration file for paths, target definitions, candidate hours, and training settings.
- [src/campaign_recommendation](/d:/aiml/campaign-recommenadtion/src/campaign_recommendation)
Main application package.
- [training-data](/d:/aiml/campaign-recommenadtion/training-data)
Input raw dataset.

- [outputs](/d:/aiml/campaign-recommenadtion/outputs)
Generated model artifacts, recommendations, summaries, and explanations.
- [reports](/d:/aiml/campaign-recommenadtion/reports)
Benchmark and comparison reports.

4. Input Data Used

Training dataset:

-
[mfl_last_6_months_communication_data.csv](/d:/aiml/campaign-recommenadtion/training-data/mfl_last_6_months_communication_data.csv)

Important raw columns used:

- 'apac_card_number'
- 'comm_status'
- 'communication_type'
- 'payment_unique_id'
- 'created_date'
- 'risk'
- 'collectable_amount'
- 'campaign_identifier'
- 'emi_date'

5. Data Filtering Rules

EMI Day Filter

Only rows where 'emi_date' is the '5th' are used.

This is controlled by:

- 'emi_day_filter = 5' in
[default_config.json](/d:/aiml/campaign-recommenadtion/config/default_config.json)

Allowed Strategy Days

Only the following day offsets are considered:

- '-5, -4, -3, -2, -1, 1, 2, 3, 4, 5'

'D' or 'day_offset = 0' is excluded from campaign generation.

6. Target Label Design

Binary Success

The model still trains as a classifier with binary success target:

- 'target_success = 1' for positive communication outcome
- 'target_success = 0' otherwise

Channel-Aware Success

Success is defined differently by channel:

- 'SMS': 'DELIVERED', 'READ', 'CLICKED'
- 'WHATSAPP': 'DELIVERED', 'READ', 'CLICKED'
- 'VOICE': 'CONNECTED', 'CALL_CONNECTED'

This logic is implemented in:

- [labeling.py](/d:/aiml/campaign-recommenadtion/src/campaign_recommendation/labeling.py)

Weighted Success Strength

To make stronger engagement more important:

- 'DELIVERED = 0.4'
- 'READ = 0.8'
- 'CLICKED = 1.0'
- 'CONNECTED / CALL_CONNECTED = 1.0'

This creates:

- 'target_success_score'
- 'target_sample_weight'

The classifier still predicts binary success probability, but stronger events influence training more.

7. Feature Engineering

Feature engineering is implemented in:

- [features.py](/d:/aiml/campaign-recommenadtion/src/campaign_recommendation/features.py)
- [data.py](/d:/aiml/campaign-recommenadtion/src/campaign_recommendation/data.py)

Base Features

- 'risk'
- 'communication_type'
- 'campaign_identifier'
- 'day_offset'
- 'send_hour'
- 'collectable_amount'
- 'emi_day'
- 'emi_month_number'
- 'is_predue'
- 'is_postdue'

Previous-Month History Features

For each customer, previous-month aggregates are created:

- 'prev_month_total_comms'
- 'prev_month_success_count'
- 'prev_month_payment_count'
- 'prev_month_sms_count'
- 'prev_month_whatsapp_count'
- 'prev_month_voice_count'
- 'prev_month_sms_success_count'
- 'prev_month_whatsapp_success_count'
- 'prev_month_voice_success_count'
- 'prev_month_success_rate'
- 'prev_month_payment_rate'

Previous-Month Day Availability

These indicate whether a customer had campaign activity in the previous month for that specific EMI-relative day:

- 'prev_month_day_m5_available'
- 'prev_month_day_m4_available'
- 'prev_month_day_m3_available'
- 'prev_month_day_m2_available'
- 'prev_month_day_m1_available'
- 'prev_month_day_p1_available'
- 'prev_month_day_p2_available'
- 'prev_month_day_p3_available'
- 'prev_month_day_p4_available'
- 'prev_month_day_p5_available'

And a runtime helper feature:

- 'prev_month_same_day_available'

This is used to decide whether a day should produce recommendations or '-'.

8. Train/Validation/Test Split

The split is time-based, not random.

This is important because recommendation systems for EMI communication must be validated on future months, not mixed-row random samples.

Current split logic:

- train = all older months except last two
- validation = second latest EMI month
- test = latest EMI month

Example from current runs:

- train: '2025-10', '2025-11', '2025-12'
- validation: '2026-01'
- test: '2026-02'

9. Model Selection Journey

Initial Legacy Baseline

The first production baseline used logistic regression with calibrated probabilities.

Weighted Success Challenger

Then channel-aware weighted success was introduced.

Tree-Based Experiment Benchmark

Multiple experiments were run:

- logistic baseline
- random forest
- extra trees
- histogram gradient boosting
- adaboost

Benchmark reports:

-
[model_benchmark_experiments.csv](/d:/aiml/campaign-recommenadtion/reports/model_benchmark_experiments.csv)
-
[model_benchmark_experiments.md](/d:/aiml/campaign-recommenadtion/reports/model_benchmark_experiments.md)

Final Current Default

The current default production model is:

- 'ExtraTreesClassifier'

This decision was based on stronger benchmark results against the legacy model.

Decision report:

- [legacy_vs_extra_trees.md](/d:/aiml/campaign-recommenadtion/reports/legacy_vs_extra_trees.md)

10. Current Production Model Implementation

Main model code:

- [modeling.py](/d:/aiml/campaign-recommenadtion/src/campaign_recommendation/modeling.py)

Current production training stack:

1. Feature engineering
2. Preprocessing using 'ColumnTransformer'
3. 'ExtraTreesClassifier'

4. Probability calibration using logistic regression on validation predictions

11. Where The Model Artifact Is Stored

The trained model artifact is stored in:

- [outputs/model/model.joblib](/d:/aiml/campaign-recommenadtion/outputs/model/model.joblib)

Important:

- this is the primary serialized model artifact
- it is a 'joblib' file, not a '.pkl'
- functionally it serves the same deployment purpose as a pickle-style model artifact

Related saved artifacts:

- [outputs/model/metrics.json](/d:/aiml/campaign-recommenadtion/outputs/model/metrics.json)
- [outputs/model/run_summary.json](/d:/aiml/campaign-recommenadtion/outputs/model/run_summary.json)
- [outputs/model/evaluation_sample.csv](/d:/aiml/campaign-recommenadtion/outputs/model/evaluation_sample.csv)
- [outputs/model/validation_slice_report.csv](/d:/aiml/campaign-recommenadtion/outputs/model/validation_slice_report.csv)
- [outputs/model/calibration_report.csv](/d:/aiml/campaign-recommenadtion/outputs/model/calibration_report.csv)

12. How Training Works

Training entrypoint:

- [__main__.py](/d:/aiml/campaign-recommenadtion/src/campaign_recommendation/__main__.py)

Training pipeline:

- [pipelines/train.py](/d:/aiml/campaign-recommenadtion/src/campaign_recommendation/pipelines/train.py)

Command:

```
'''powershell
$env:PYTHONPATH="src"
d:\aiml\venv\Scripts\python.exe -m campaign_recommendation train --max-rows 80000
'''
```

What happens during training:

1. Load config
2. Read CSV in chunks

3. Filter only EMI day 5 rows
4. Build target labels and weights
5. Build previous-month aggregates
6. Build time-based splits
7. Train Extra Trees classifier
8. Fit probability calibrator
9. Evaluate on validation and test
10. Save model and metrics

13. How Inference Works

There are two main inference modes.

A. Batch Recommendation Generation

Command:

```
'''powershell
$env:PYTHONPATH="src"
d:\aiml\venv\Scripts\python.exe -m campaign_recommendation recommend --model-dir outputs\mo
'''
```

Pipeline:

```
-
[pipelines/recommend.py](/d:/aiml/campaign-recommenadtion/src/campaign_recommendation/pipel
/recommend.py)
- [recommend.py](/d:/aiml/campaign-recommenadtion/src/campaign_recommendation/recommend.py)
```

What it does:

1. Load 'model.joblib'
2. Build base population
3. Generate candidate communications for all allowed days, channels, and configured hours
4. Score every candidate with calibrated model probability
5. Apply per-day quota by risk
6. Insert blank '-' rows for unavailable days
7. Export strategy outputs

B. Single Account Explanation

Command:

```
'''powershell
$env:PYTHONPATH="src"
d:\aiml\venv\Scripts\python.exe -m campaign_recommendation explain --model-dir outputs\mode
--account-id MFLAPDSECUL000005010246
'''
```

What it exports:

- day-level best scores
- final selected recommendations

- strategy calendar
- top contributing features for the top recommendation

14. What Input Is Needed For Inference

If you already have a prepared account file for current-period inference, the expected fields are:

- 'apac_card_number'
- 'risk'
- 'collectable_amount' or 'outstanding_balance'
- 'campaign_identifier'
- 'emi_date'

Optional but useful if available:

- previous-month feature columns
- day availability columns

If current-state features like 'POS', 'bucket', or additional account descriptors are later added to training data, they can also be added to inference.

15. Recommendation Output Files

Main Strategy Output

- [outputs/recommendations.csv](/d:/aiml/campaign-recommenadtion/outputs/recommendations.csv)

This contains the day-by-day strategy calendar with:

- account
- risk
- day
- slot rank
- strategy label
- availability flag

Bucket Summaries

- [outputs/bucket_counts.csv](/d:/aiml/campaign-recommenadtion/outputs/bucket_counts.csv)
- [outputs/bucket_counts_by_day.csv](/d:/aiml/campaign-recommenadtion/outputs/bucket_counts_by_day.csv)

These summarize:

- count by 'risk + strategy label'
- count by 'risk + day + strategy label'

Account-Level Explanation Outputs

Examples:


```
-
[MFLAPDSECUL000005010246_day_scores.csv](/d:/aiml/campaign-recommenadtion/outputs/explanati
MFLAPDSECUL000005010246_day_scores.csv)
-
[MFLAPDSECUL000005010246_final_recommendation.csv](/d:/aiml/campaign-recommenadtion/outputs/
lanations/MFLAPDSECUL000005010246_final_recommendation.csv)
-
[MFLAPDSECUL000005010246_strategy_calendar.csv](/d:/aiml/campaign-recommenadtion/outputs/ex
ations/MFLAPDSECUL000005010246_strategy_calendar.csv)
```

16. How The Final Strategy Is Built

This is implemented in:

```
- [recommend.py](/d:/aiml/campaign-recommenadtion/src/campaign_recommendation/recommend.py)
```

Rule flow:

1. Generate candidates for each account and each day
2. Predict success probability
3. Rank by probability inside the day
4. Apply day-wise quota by risk
5. If the previous month had no activity for that day, return '-'

So the final calendar is not just top-N overall. It is top-N **per day**.

17. Explainability

Explainability code:

```
- [explain.py](/d:/aiml/campaign-recommenadtion/src/campaign_recommendation/explain.py)
```

What is supported:

- top contributing features for a chosen prediction
- validation slice reporting by:
 - split
 - risk
 - communication type
 - day offset
- calibration report

How explanation is calculated:

1. Reuse model preprocessor
2. Transform candidate row
3. Use underlying model feature contributions
4. Rank highest absolute contributions

18. Deployment Notes

Current implementation is a CLI-first batch system.

Current Deployable Pattern

The simplest deployment pattern is:

1. Train offline on historical batch data
2. Store 'model.joblib'
3. Run batch recommendation inference on current EMI population
4. Export CSV outputs for campaign operations

If Exposing As An API Later

Recommended API flow:

1. Load 'outputs/model/model.joblib' at service startup
2. Accept account input payload
3. Build candidate day/channel/hour rows
4. Score candidates
5. Apply per-day quota logic
6. Return JSON calendar and summaries

Example Inference Service Contract

Input:

- account id
- emi date
- risk
- outstanding balance
- campaign identifier
- previous-month aggregates if externally precomputed

Output:

- daily strategy calendar
- selected communications per day
- optional explanation payload

19. GitHub And Versioning

The project was pushed to:

- 'https://github.com/Sumitkumar1007/campaign-recommendation.git'

Repository setup includes:

- Git LFS for the large training CSV
- main branch push completed

Large file tracking:

-

[training-data/mfl_last_6_months_communication_data.csv](/d:/aiml/campaign-recommenadtion/t
ing-data/mfl_last_6_months_communication_data.csv) is tracked through Git LFS

20. How To Reproduce End To End

Step 1: Activate environment

```
'''powershell
$env:PYTHONPATH="src"
'''
```

Step 2: Train model

```
'''powershell
d:\aiml\venv\Scripts\python.exe -m campaign_recommendation train --max-rows 80000
'''
```

Step 3: Generate recommendations

```
'''powershell
d:\aiml\venv\Scripts\python.exe -m campaign_recommendation recommend --model-dir outputs\mo
'''
```

Step 4: Explain single account

```
'''powershell
d:\aiml\venv\Scripts\python.exe -m campaign_recommendation explain --model-dir outputs\mode
--account-id MFLAPDSECUL0000005010246
'''
```

Step 5: Benchmark models

```
'''powershell
d:\aiml\venv\Scripts\python.exe -m campaign_recommendation benchmark-models --max-rows 8000
'''
```

21. Current Limitations

- current inference mostly relies on communication history and account-level risk/balance context
- richer variables like 'POS', 'bucket', and more loan features are not yet present in the training file
- candidate send hours are currently config-driven
- recommendation generation is batch-first and can be slow on large volumes

22. Recommended Next Steps

1. Add 'POS', 'bucket', and other current-state loan features if available in training data
2. Persist precomputed previous-month aggregates for faster inference.
3. Add API serving layer for real-time or on-demand scoring.
4. Add pivoted business report output in Excel-style calendar format.
5. Add experiment registry for model lineage and approval workflow.

23. Final Summary

This project now supports:

- EMI-specific campaign recommendation
- day-wise communication quotas by risk
- blank-day handling based on previous-month availability
- channel-aware success logic
- weighted engagement-aware training
- benchmarked model selection
- Extra Trees as the current production default
- batch inference and account-level explanation
- saved model artifact and reproducible CLI workflow